

GSEA-Agent: A Planning-and-Decomposition Agent for Frontier-Market Equity Analysis, and the Hardware Physics of Long-Context Prefill versus Dynamic Tool-Use on Cloud TPU v5e

Bennet Brown
CSE 590 A Systems for Machine Learning
bnb2122@uw.edu

June 11, 2026

Abstract

We present **GSEA-Agent**, a planning-and-decomposition agent that automates the analysis of Ghana Stock Exchange (GSE) financial filings, turning unformatted, often scanned PDF balance sheets into structured equity and credit-risk assessments. The agent is a multi-tiered LangGraph state machine: a *Macro-Planner* emits JSON-constrained sub-tasks, a *Perception and Tabular Inversion* layer wraps **marker-pdf** to recover Markdown tables, and a *Synthesis Engine* composes the final report. Against a single open-weights model (**Qwen3-4B**) served by vLLM on a Cloud TPU v5e VM, we evaluate two contrasting context-delivery strategies, plus an agentic variant: **(A)** naïve long-context prefill, which appends the entire document into one prompt; **(B)** directive RAG, which chunks and retrieves only the top- k relevant blocks per sub-task; and **(C)** interleaved RAG, which retrieves *between* generation turns. We profile time-to-first-token (TTFT), inter-token latency (ITL), end-to-end wall-clock, decode/prefill throughput, and on-chip KV-cache (HBM) utilization across controlled context-scaling factors and a multi-filing corpus. Our measurements test two structural hypotheses: **(I)** that long-context prefill is dominated by the $O(N^2)$ self-attention cost and a growing decode-time KV read penalty, and **(II)** that tool-use orchestration introduces a “latency bubble” of idle retrieval and re-prefill time when control returns to the host. We close with a roofline / arithmetic-intensity analysis showing autoregressive decode is memory-bandwidth-bound (near 1 FLOP/byte on the weight stream) and propose concrete TPU mitigations: automatic prefix caching, chunked prefills, and asynchronous extraction queues.

1 Introduction

1.1 Information Barriers in Frontier Equity Markets

Frontier and emerging equity markets such as the Ghana Stock Exchange (GSE) operate under acute information friction. Material disclosures—quarterly and annual financial statements, unaudited summary results, and qualitative management commentary—are published as press-release PDFs whose tables are frequently rasterized scans rather than machine-readable text. In a time-sensitive trading environment, the analytical pipeline is bottlenecked by reasoning, and more importantly, by *ingestion*: an analyst must manually transcribe balance-sheet line items, reconcile them across periods, and read narrative risk commentary before any decision can be made. This manual extraction step is slow, error-prone, and does not scale across the dozens of issuers that report within a narrow window.

An autonomous agent that reads these filings end-to-end—recovering tabular structure, decomposing an analyst’s request into verifiable sub-tasks, and synthesizing a structured assessment—directly attacks this barrier. But deploying such an agent on cost-constrained accelerators raises a systems question that is the focus of this paper: *how should the document be delivered to the model?* The naïve answer—put the whole filing in the prompt—interacts poorly with the quadratic cost of self-attention and the finite High Bandwidth Memory (HBM) of a single accelerator. The alternative—retrieval-augmented generation (RAG)—trades that prefill cost for orchestration overhead and repeated host round-trips. Quantifying this trade-off on real hardware is our goal.

1.2 Contributions

This paper makes the following contributions:

- **The GSEA-Agent application.** A planning-and-decomposition agent (Section 2) that converts unstructured GSE filings into structured equity/credit analyses, built as a LangGraph state machine over a TPU-served open-weights LLM.
- **A controlled systems benchmark** (Section 3) contrasting three context-delivery mechanisms—long-context prefill (A), directive RAG (B), and interleaved/agentive RAG (C)—under identical task plans, with an open-source profiling harness that records TTFT, ITL, end-to-end latency, throughput, accuracy, and KV-cache HBM utilization across context-scaling factors and multiple real filings.
- **Two structural hypotheses, tested empirically** (Section 4). *Hypothesis I*: long-context prefill is governed by the $O(N^2)$ self-attention term, producing a super-linear TTFT curve, a linear rise in ITL as the resident KV cache grows, and a matching HBM footprint. *Hypothesis II*: tool-use orchestration creates a measurable “latency bubble”—idle accelerator time spent in retrieval and re-prefill when the host regains control between turns.
- **A systems-level optimization analysis** (Section 5) that models decode as a bandwidth-bound regime via arithmetic intensity and roofline reasoning, and maps concrete TPU mitigations (automatic prefix caching, chunked prefills, asynchronous extraction) onto the measured bottlenecks.

2 GSEA-Agent Application Architecture

2.1 The Planning-and-Decomposition Design Pattern

A single-prompt wrapper conflates planning, retrieval, extraction, and synthesis in one context window. GSEA-Agent uses a **planning-and-decomposition** LangGraph state machine: decompose into sub-tasks, resolve each against controlled context, then compose results. Context delivery (Mechanisms A/B/C) is swappable without changing the rest of the graph.

2.2 Component Topology

Macro-Planner. The planner ingests the analyst’s natural-language query and emits a *structurally constrained* JSON object enumerating isolated sub-tasks (e.g., “compute liquidity metrics”, “summarize management risk commentary”, “extract total assets and total equity”). JSON constraints make the plan machine-checkable and give the downstream nodes a deterministic schedule rather than a free-form chain of thought.

Perception and Tabular Inversion Layer. GSE filings arrive as PDFs whose financial tables are often scanned images. This layer wraps **marker-pdf** (which uses the Surya family of layout/OCR/table-recognition models) as a tool that downloads a filing and inverts it into unified Markdown, preserving tabular structure so that line items remain associated with their period columns. Because this step is CPU-heavy and runs as a subprocess, it is the natural *host orchestration* cost we later argue should be hidden behind asynchronous queues (Section 5). The extracted Markdown is cached on disk and is re-generated on demand from the source URL, so the ingestion cost can be measured as part of the true end-to-end flow.

Retrieval Tool (RAG). For Mechanisms B and C, a retrieval tool ingests the Markdown, splits it with a *table-preserving* chunker (large chunks with overlap so balance-sheet tables are not fragmented), filters low-signal fragments, and de-duplicates repeated blocks. Chunks are embedded with a sentence-transformer encoder and indexed in FAISS. Retrieval is *hybrid*: dense FAISS vectors fused with TF-IDF lexical scores via weighted reciprocal-rank fusion (lexical up-weighted for exact-figure queries on dense tables).

Synthesis Engine. The final stage gathers the compiled matrices and management notes and performs the analytical reasoning—liquidity and profitability changes, cash-flow position, and risk-strategy assessment—emitting the structured report returned to the analyst. The engine is invoked once per single-pass mechanism (A, B) and once per sub-task turn in the interleaved mechanism (C), which is what exposes Hypothesis II.

3 Experimental Methodology and Infrastructure

3.1 Hardware and Serving Topology

All experiments use a single open-weights model, **Qwen3-4B**, served by the vLLM execution engine on a Google Cloud **TPU v5e** virtual machine (a single-chip **v5litepod-1** slice with 16 GB of HBM). The client and the agent communicate with the server over its OpenAI-compatible HTTP API. Because a **v5litepod-1** exposes a single TPU chip, the serving engine and any standalone profiler contend for the same device; our live profiling therefore attaches to the running server rather than spawning a second engine. The profiling harness auto-detects the served model, performs a warmup request to absorb one-time XLA compilation, and records one tidy CSV row per request. All measurements reported in this paper were collected in **June 2026** on the course TPU v5e VM, consistent with the project proposal timeline (May–June 2026).

3.2 Controlled Variables

We sweep three independent axes. **Context size** is varied by a duplication factor $x \in \{1, 2, 4, 6\}$ applied to a base filing, isolating the effect of prompt length N_{prefill} under otherwise identical content. **Concurrency** is varied to expose throughput saturation. **Document** is varied across a multi-filing corpus (Access Bank Q1 2026 and Fan Milk FY2025) declared in a registry so each filing is re-extracted from its source URL and every metric row is tagged with its document of origin. The task plan (sub-tasks), system prompt, and output token cap are held fixed across mechanisms so latency is comparable.

3.3 Mechanism A: Naïve Long-Context Prefill

The agent appends the complete extracted Markdown document directly into a single-pass active prompt context window and decodes one response. This is the upper bound on prefill cost: the full filing is materialized as keys and values in HBM, and self-attention is computed over the entire sequence before the first token is produced.

3.4 Mechanism B: Directive RAG Tool Orchestration

The document is chunked, embedded, and indexed; the vector store is treated as an explicit tool queried once *up front* for every sub-task, and the union of retrieved blocks forms a compact context for a single decode pass. This bounds prompt length independently of document size at the cost of an ingestion phase (chunk + embed + FAISS sync) and possible recall loss.

3.5 Mechanism C: Interleaved (Agentic) RAG

To probe Hypothesis II we add an interleaved variant that retrieves *between* generation turns: for each sub-task the agent retrieves, generates a turn, then resumes. On a stateless serving API each resume re-prefills the grown conversation and each retrieval is idle accelerator time. Mechanisms B and C share the same plan and the same retrieved chunks; the only difference is *when* retrieval happens, which isolates the orchestration overhead.

3.6 Metrics

Per request we log: prompt (prefill) tokens N_{prefill} , completion tokens, **TTFT**, **ITL**, end-to-end wall-clock, prefill and decode throughput (tok/s), and **peak KV-cache utilization** scraped from the server’s metrics endpoint as our *HBM activation-footprint proxy*. For Mechanism B/C we additionally log the RAG setup breakdown (chunk/embed/index and per-query search time) and, for C, the interleaving bubble metrics (idle retrieval gap, re-prefill on resume, active decode time, and

Config	N_{prefill}	TTFT (s)	E2E (s)	HBM (%)
A, $x1$	4,359	0.038	0.862	8.8
A, $x2$	8,642	0.570	1.442	17.1
A, $x4$	17,208	1.412	2.382	34.0
A, $x6$	25,774	2.539	3.606	50.9
B, $x1$	2,229	0.137	0.927	4.5
B, $x2$	2,285	0.124	0.915	4.7
B, $x4$	2,336	0.099	0.890	4.8
B, $x6$	2,336	0.043	0.834	4.8

Table 1: Core latency and memory metrics vs. context scaling (Access Bank, concurrency = 1, output cap 64 tokens). HBM (%) is peak KV-cache utilization. As context grows $6\times$, Mechanism A’s TTFT rises $\sim 67\times$ and its KV footprint $\sim 6\times$, while Mechanism B stays bounded because retrieved context length is fixed. Source: `metrics_output/results.csv`.

utilization = decode/wall). A separate accuracy harness scores answers against per-filing ground-truth figures, adding a quality axis to the latency sweep. Upstream, we record Macro-Planner JSON latency and **marker-pdf** extraction wall-clock (download vs. parse) as the host-orchestration baseline.

4 Empirical Evaluation and Metric Breakdown

All numbers below are measured on **Qwen3-4B** served by vLLM on the TPU v5e VM. Unless stated otherwise, the primary document is the Access Bank Q1 2026 filing; the Fan Milk FY2025 filing is used for the cross-document comparison.

4.1 Headline Results

Table 1 reports the core metrics for Mechanisms A and B across the scaling factors (Access Bank, concurrency = 1); Mechanism C’s interleaving bubble is reported in Table 2.

4.2 Upstream Processing Baseline

Before inference, the host pays a one-time orchestration cost. The Macro-Planner emits its JSON sub-task plan in 1.06 s (≈ 86 output tokens, TTFT 0.026 s). **marker-pdf** extraction of the Fan Milk filing took 0.71 s to download + 69.4 s to parse (70.1 s total for a 12.6k-character Markdown). The denser, single-page Access Bank statement is dramatically more expensive to OCR: 0.73 s download + 1347 s parse (1348 s total for a 26.7k-character Markdown), i.e. $\sim 19\times$ longer on the Marker pass despite fewer pages, because Surya processes every cell of a dense multi-section balance sheet. This host-CPU phase is pure TPU idle time, motivating the asynchronous queue in Section 5. Source: `metrics_output/extraction.csv`.

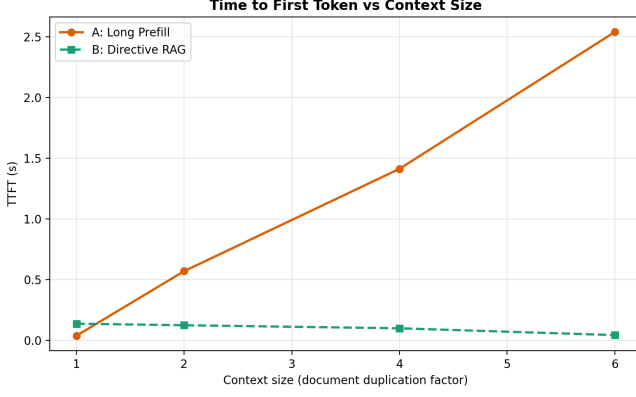


Figure 1: TTFT vs. context size, Mechanism A vs. B (Access Bank, concurrency = 1). A grows super-linearly with N_{prefill} ; B stays approximately flat because retrieved context length is bounded.

4.3 Time to First Token (TTFT)

TTFT isolates the prompt-evaluation (prefill) penalty. TTFT_A rises sharply with x while TTFT_B remains near-constant (Fig. 1). The headline ratio is $\text{TTFT}_A(x6)/\text{TTFT}_A(x1) \approx 67\times$ for only a $5.9\times$ growth in N_{prefill} . Even discounting the unusually low $x1$ point (0.038 s, likely favoured by prefix reuse from warmup), TTFT still rises $4.5\times$ ($0.570 \rightarrow 2.539$ s) from $x2$ to $x6$ against a $3.0\times$ token increase—clearly super-linear, the headline evidence for Hypothesis I.

4.4 Inter-Token Latency (ITL)

ITL maps the autoregressive decode phase. Each generated token requires streaming the full model weight matrix from HBM and reading the entire resident KV cache for the causal attention pass. Because decode is memory-bandwidth-bound (Section 5), any growth in KV footprint directly increases per-token HBM traffic and therefore ITL. Mechanism A materializes the full duplicated document in KV storage; Mechanism B keeps a compact, retrieval-bounded context. The measured curves (Fig. 2) reflect this physics: ITL_A rises monotonically with context factor while ITL_B stays flat.

On Access Bank (concurrency = 1), ITL_A grows from 13.1 to 16.9 ms/token ($x1 \rightarrow x6$, +29%), whereas ITL_B holds near 12.6 ms/token across all factors—a $1.35\times$ gap at $x6$. Fan Milk shows the same pattern (12.8 $\rightarrow$ 15.5 ms for A vs. 12.4 ms flat for B; $1.25\times$ at $x6$). Decode throughput mirrors the ITL split: Mechanism A falls from 77.6 to 59.9 tok/s while Mechanism B stays near 81 tok/s. Mechanism A therefore trades a massive TTFT penalty and a sustained decode slowdown for the benefit of a richer single-pass context; Mechanism B preserves decode speed by bounding KV size.

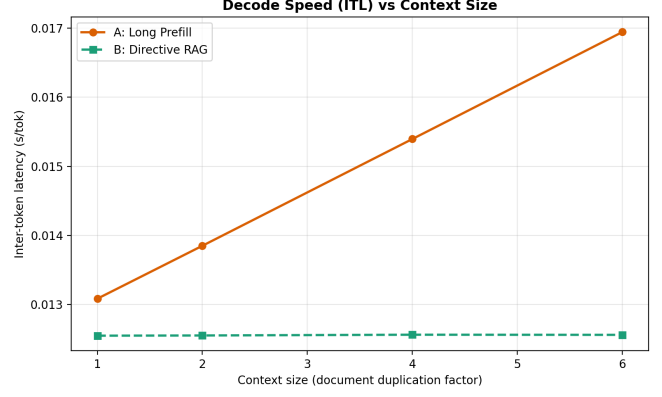


Figure 2: Inter-token latency vs. context size (Access Bank). Mechanism A exhibits a linear upward trajectory as KV-cache HBM traffic grows; Mechanism B remains flat because retrieved context length is bounded.

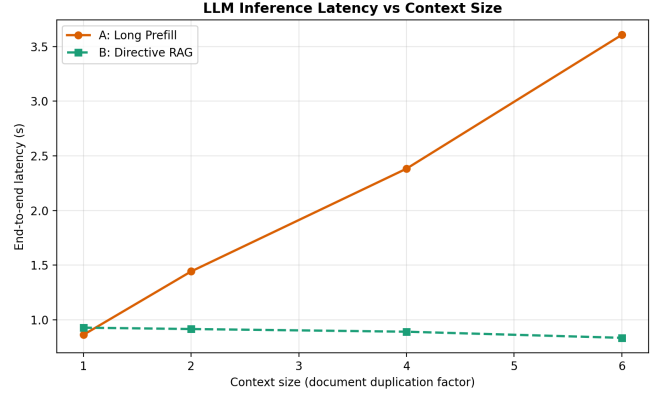


Figure 3: End-to-end latency vs. context size, A vs. B (Access Bank). B’s total is flat in document size; A’s grows with prefill.

4.5 End-to-End Latency

End-to-end wall-clock aggregates prefill, decode, and (for B) retrieval across the full graph. Fig. 3 contrasts the totals: at $x1$ the two are comparable (A 0.86 s vs. B 0.93 s, B paying its one-time index/search overhead), but B overtakes A from $x2$ onward and the gap widens to $4.3\times$ at $x6$ (A 3.61 s vs. B 0.83 s). The crossover sits between $x1$ and $x2$.

4.6 HBM Memory Footprint

Peak KV-cache utilization (Fig. 4) is our device-memory evidence: Mechanism A’s footprint grows almost linearly with prompt length, from 8.8% at $x1$ to 50.9% at $x6$ (a $5.8\times$ increase tracking the $5.9\times$ token growth), whereas Mechanism B stays bounded near 4.5–4.8%. Extrapolating A’s trend, the single 16 GB chip would saturate its KV cache at roughly $x12$, an explicit out-of-memory ceiling that B never approaches.

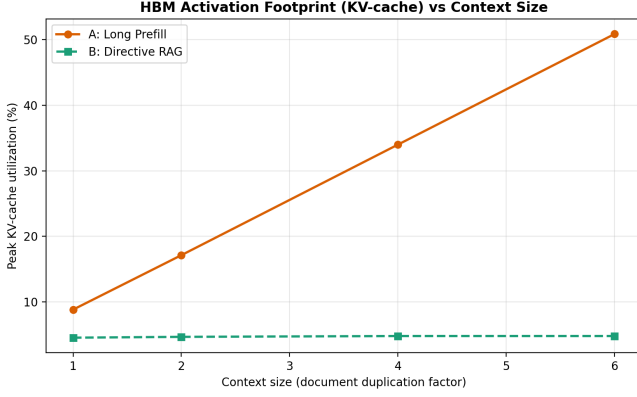


Figure 4: Peak KV-cache (HBM activation) utilization vs. context size (Access Bank). A’s resident KV footprint scales with N_{prefill} ; B remains bounded.

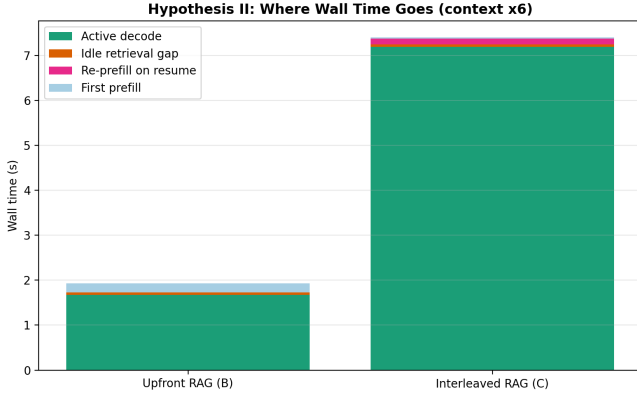


Figure 5: Wall-time decomposition (Access Bank, $x6$): active decode, idle retrieval gap, re-prefill on resume, and first prefill, for upfront (B) vs. interleaved (C). Only C incurs a re-prefill term.

4.7 Hypothesis I

Figures 1–4 and Table 1 confirm Hypothesis I: TTFT grows super-linearly in N_{prefill} , ITL rises for Mechanism A only (+29%, $x1 \rightarrow x6$), and HBM scales to 50.9% at $x6$ while Mechanism B stays near 4.8%.

4.8 Hypothesis II: the RAG Orchestration Gap

Mechanism C incurs idle retrieval and a re-prefill penalty of 0.13–0.44 s per run (Table 2, Fig. 5); wall-clock is $\sim 3.8\times$ Mechanism B. The naïve utilization = decode/wall ratio is *higher* for C (93–97% vs. 81–87%) because the five-turn pattern emits $\sim 4.6\times$ more tokens—the true cost is the *re-prefill tax*, which shrinks from 0.44 s at $x1$ to 0.13 s at $x6$ (prefix reuse).

Pattern	Decode (s)	Idle (s)	Re-prefill (s)	Wall (s)	Util. (%)
Upfront (B)	1.675	0.050	0.000	1.933	86.6
Interleaved (C)	7.193	0.053	0.128	7.401	97.2

Table 2: Hypothesis II wall-time decomposition (Access Bank, $x6$, 5 sub-tasks). C emits $\sim 4.6\times$ more tokens than B (585 vs. 128), so its higher utilization reflects more decode work, not less overhead; the orchestration cost surfaces as the re-prefill term and the $3.8\times$ wall-clock. Source: `metrics_output/interleaved.csv`.

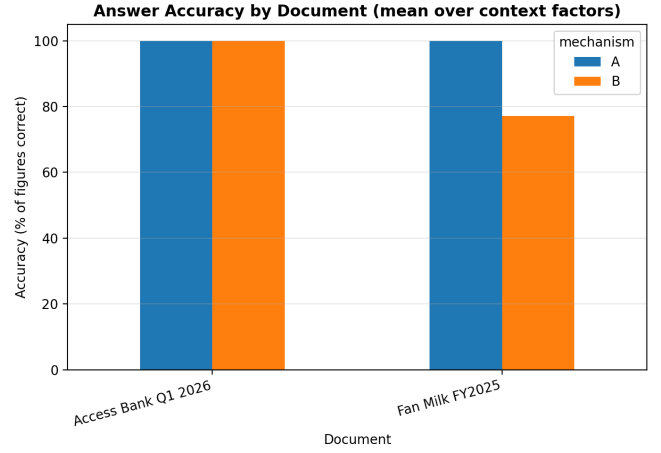


Figure 6: Exact-figure recall by document and mechanism (mean over context factors). A is perfect on both; B matches A on Access Bank but loses recall on the denser Fan Milk tables.

4.9 Quality vs. Speed and Cross-Document View

Beyond latency, the accuracy harness scores exact-figure recall against per-filing ground truth (Fig. 6). Mechanism A (full context) answers *all* questions correctly on both filings and at every scaling factor (100%), confirming that when the figure is in the prompt the model reliably extracts it. Mechanism B (RAG) is perfect on Access Bank (100%) but drops to 77% on Fan Milk (83% at $x1$, 75% at $x2$ – $x6$), where a handful of densely-tabulated line items are missed by retrieval. This is the core quality-vs-speed trade-off: B is up to $4.3\times$ faster and uses $\sim 10\times$ less HBM, but cedes up to 25 points of recall on the harder document. Fig. 7 shows the corresponding end-to-end latency comparison across both filings.

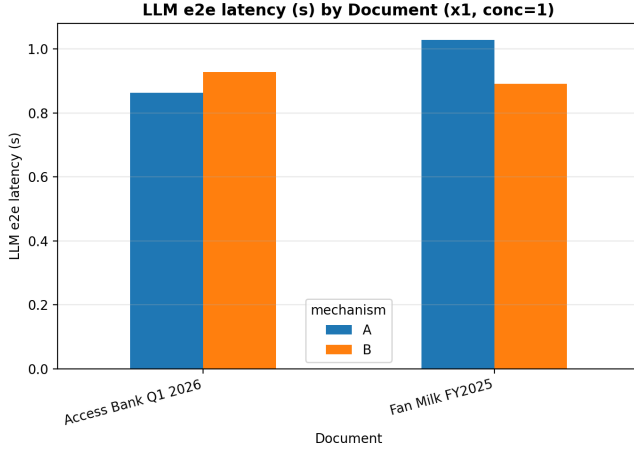


Figure 7: End-to-end latency by document and mechanism ($x1$, concurrency = 1). The A-vs-B ordering is consistent across filings.

5 Systems-Level Optimization Analysis

5.1 Arithmetic-Intensity Modeling of Decode

We model decode cost with two quantities: P , the model’s total parameter count, and B_{KV} , the bytes of key-value cache data read from HBM for each autoregressively generated token. B_{KV} grows with the prefilled context length N_{prefill} —Mechanism A materializes the full document in KV storage, so B_{KV} rises with context factor; Mechanism B bounds prompt length and keeps B_{KV} small.

The two phases of LLM inference have opposite operational profiles. *Prefill* processes all N_{prefill} tokens in parallel; it performs on the order of $2PN_{\text{prefill}}$ matrix-multiply FLOPs plus the $O(N_{\text{prefill}}^2)$ attention term, reusing each loaded weight across many tokens—a *compute-bound* regime of high arithmetic intensity. *Decode*, by contrast, generates one token at a time ($b = 1$ per request). Per decoded token the engine streams all P parameters from HBM once, performing $\approx 2P$ FLOPs while reading $\approx 2P$ bytes of weights (bf16, 2 bytes per parameter), plus B_{KV} for the causal attention pass. The decode arithmetic intensity is therefore

$$I_{\text{decode}} = \frac{\text{FLOPs}}{\text{HBM bytes accessed}} = \frac{2P}{2P + B_{KV}} \approx 1 \text{ FLOP/byte when } B_{KV} \ll 2P, \quad (1)$$

For Qwen3-4B ($P \approx 4 \times 10^9$), the weight footprint alone is ≈ 8 GB in bf16; at our largest Mechanism A context ($x6$), peak KV-cache utilization reaches 50.9% of the 16 GB chip—adding substantial B_{KV} that Mechanism B (bounded near 4.8%) never pays. Larger B_{KV} pushes I_{decode} further into the bandwidth-bound regime (Section 4).

On a roofline for TPU v5e—whose ridge point is peak

bf16 throughput (≈ 197 TFLOP/s) divided by HBM bandwidth (≈ 819 GB/s), i.e. ≈ 240 FLOP/byte—even the weight-only intensity (≈ 1 FLOP/byte) sits far to the left of the ridge: single-stream decode is **memory-bandwidth-bound**, and the accelerator’s matrix units are largely idle during generation. Mechanism A’s larger B_{KV} pushes intensity further into the bandwidth-bound regime, which is why *batching*, *KV-cache reduction* (Mechanism B), and *prefix caching*—not faster matmul alone—are the levers that matter for decode.

5.2 Architectural Mitigations

The measured bottlenecks map onto three concrete, well-understood TPU optimizations.

Automatic Prefix Caching (APC). vLLM’s APC stores the KV pairs of shared prompt prefixes in a block table so that subsequent requests with the same prefix skip redundant prefill. On our stack this is enabled at serve time via `-enable-prefix-caching`. GSE filings share large static prefixes—identical statement headings, boilerplate, and (under our context-scaling harness) repeated document blocks—and analysts issue many queries against the same filing. APC converts repeated $O(N^2)$ prefill work into a one-time cost: a new sub-task reuses cached keys/values and only prefills its novel suffix. This directly attacks Hypothesis I’s TTFT curve and the re-prefill component of the Hypothesis II bubble (where we already observe the tax shrinking from 0.44 s at $x1$ to 0.13 s at $x6$, consistent with prefix reuse over duplicated content).

Chunked Prefills. vLLM’s scheduler can split a long prefill into smaller token blocks and interleave those blocks with ongoing decode batches in the same iteration (`-enable-chunked-prefill` on the serve command), rather than holding the accelerator idle until one monolithic prefill completes. This co-locates prefill (compute-bound, high I_{prefill}) with decode (bandwidth-bound, low I_{decode}), raising aggregate chip utilization and smoothing the TTFT spikes that Mechanism A exhibits at high x . For our agent, enabling chunked prefills alongside APC is the natural next experiment: the Macro-Planner’s JSON plan and the static filing header form a reusable prefix, while each sub-task’s retrieved evidence is a short suffix prefilled in chunks.

Asynchronous Execution Queues. The CPU-heavy `marker-pdf` layout parser runs as a host subprocess during which the TPU sits idle (quantified in the upstream baseline, Section 4). Moving extraction onto background worker threads / a queue—so ingestion of the next filing overlaps inference on the current one—hides this host-orchestration downtime and keeps the accelerator busy, which is especially valuable in the batch-reporting window when many issuers file at once.

6 Related Work, Conclusion, and Future Work

Related Work. Our serving stack builds on vLLM and its PagedAttention KV-cache management [1]; the long-context cost we measure is the target of kernel-level advances such as FlashAttention [2], and the scaling of large-model execution relates to tensor/pipeline parallelism in the style of Megatron-LM [3]. Mechanism B instantiates retrieval-augmented generation [4]; relative to standard enterprise RAG frameworks, our contribution is not a new retriever but a controlled, hardware-level comparison of *when and how* context is delivered, and the agent itself is orchestrated with LangGraph. To our knowledge the interleaved-RAG “latency bubble” is rarely measured directly on accelerator hardware.

Conclusion. GSEA-Agent shows that a planning-and-decomposition design pattern can automate analytical workflows over the unstructured filings that bottleneck frontier equity markets. Our central finding is a systems trade-off: long-context prefill (Mechanism A) pays an $O(N^2)$ prefill cost, a growing KV-cache footprint, and higher ITL as decode reads scale with N_{prefill} , while dynamic tool-use (Mechanisms B/C) bounds KV size and decode speed but introduces an orchestration latency bubble (re-prefill tax) on interleaved retrieval. Neither domi-

nates unconditionally; the right policy depends on document size, query multiplicity, and the availability of prefix caching.

Future Work. (i) Exact HBM bytes via device memory instrumentation to complement the KV-cache-utilization proxy; (ii) repeated trials with $p50/p95/p99$ error bars; (iii) enabling APC and chunked prefills in the serving config and re-measuring the bubble; (iv) stronger finance-tuned embeddings to close the residual retrieval gap; and (v) extending the multi-filing corpus to a broader cross-issuer benchmark.

References

- [1] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [2] T. Dao *et al.*, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.
- [3] M. Shoeybi *et al.*, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv:1909.08053*, 2019.
- [4] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS*, 2020.